

Prefix Sum

Basics → Master: Theory, Proof, Variants & LeetCode Problem Bank

Prefix Sum is the simplest idea in this whole series — precompute running totals once, answer range queries in $O(1)$ forever after. But it quietly powers some of the hardest problems you'll see (subarray sum equals K, product of array except self, range update problems) once combined with a hashmap or extended to 2D. This guide covers the base idea fully, then goes wide on where it hides.

Section	Covers
1. What & Why	The range-sum problem, brute force -> $O(1)$ queries
2. Core Theory	Building the array, the off-by-one convention, the proof
3. Java Templates	1D prefix sum, prefix sum + hashmap, difference array, 2D
4. Problem Bank (Tiers 1-6)	24 curated problems, basics -> master
5. Fully Worked Examples	4 problems solved with full reasoning + code
6. The Variant Family	Hashmap combo, difference array, 2D, XOR prefix
7. Common Mistakes	Off-by-one, forgetting the empty-prefix entry, overflow
8. Practice Roadmap	Order to solve + spaced repetition schedule

1. What Prefix Sum Solves & Why It Works

Prefix Sum answers one question fast, repeatedly: **"what is the sum of elements between index i and index j?"** Brute force recomputes that sum by looping from i to j every single time — $O(n)$ per query, $O(n \cdot q)$ for q queries. Prefix Sum spends $O(n)$ ONCE up front to precompute running totals, then answers every future range-sum query in $O(1)$.

The core idea

Build an array `prefix[]` where `prefix[i]` = sum of all elements from index 0 to i-1 (note: NOT including index i — this convention is what makes the formula clean, see Section 2). Then the sum of any range `[left, right]` (inclusive) is just:

```
rangeSum(left, right) = prefix[right + 1] - prefix[left]
```

One subtraction. No loop. That's the entire payoff — the hard part is knowing when to reach for it and how to extend it (hashmap, 2D, difference array) for the harder problems.

How to recognize a Prefix Sum problem

- Keywords: "range sum", "subarray sum equals K", "sum between indices", "number of subarrays with sum...".
- Multiple queries against the SAME array — if you're only summing once, a simple loop is enough; prefix sum earns its keep when there are many queries or you need EVERY subarray's sum efficiently.
- "Product of array except self" and similar — prefix (and suffix) accumulation of a running value, not just sum.
- Any 2D grid version of "sum of this sub-rectangle" — that's 2D prefix sum.

Prefix Sum vs Kadane's — don't confuse them: Kadane's finds the SINGLE best contiguous subarray. Prefix Sum answers sum queries for ANY given range, or counts/finds subarrays matching a sum condition across the WHOLE array. Different job, and the two are often combined (prefix sum + hashmap solves counting problems Kadane's can't touch directly).

2. Core Theory (the convention that trips everyone up first)

2.1 Building the array — the off-by-one convention

The standard convention uses an array of size $n+1$, where $\text{prefix}[0] = 0$ (sum of ZERO elements — the empty prefix) and $\text{prefix}[i] = \text{sum of the first } i \text{ elements (indices 0 to } i-1)$.

```
nums    = [ 2,  4, -1,  3,  5]
index   =  0  1  2  3  4
```

```
prefix  = [0,  2,  6,  5,  8, 13]
index   =  0  1  2  3  4  5
```

- $\text{prefix}[0] = 0$ always — this represents 'the sum of nothing before index 0'.
- $\text{prefix}[i] = \text{prefix}[i-1] + \text{nums}[i-1]$ — each entry is the previous entry plus the ORIGINAL array's element one position back (because of the shift caused by the leading 0).
- Sum of $\text{nums}[\text{left}..\text{right}]$ inclusive = $\text{prefix}[\text{right}+1] - \text{prefix}[\text{left}]$.

2.2 Why the extra $\text{prefix}[0] = 0$ entry matters (this is the #1 source of bugs)

Without it, computing the sum of a range that starts at index 0 (e.g. $\text{nums}[0..\text{right}]$) has no valid " $\text{prefix}[\text{left}]$ " to subtract — you'd need a special case. With $\text{prefix}[0] = 0$ built in, the SAME formula ($\text{prefix}[\text{right}+1] - \text{prefix}[\text{left}]$) works uniformly for every range, including ones starting at index 0. This is why professional implementations always use the $n+1$ sized array instead of trying to avoid it.

2.3 Quick proof of the formula

- $\text{prefix}[\text{right}+1] = \text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[\text{right}]$ (sum of first $\text{right}+1$ elements).
- $\text{prefix}[\text{left}] = \text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[\text{left}-1]$ (sum of first left elements).
- Subtracting cancels out everything before index left , leaving exactly $\text{nums}[\text{left}] + \dots + \text{nums}[\text{right}]$ — the range you wanted.

3. Java Templates

3.1 Standard 1D prefix sum (range queries)

```
public class PrefixSum {
    private int[] prefix;

    public PrefixSum(int[] nums) {
        prefix = new int[nums.length + 1];
        prefix[0] = 0;
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }

    // inclusive range [left, right]
    public int rangeSum(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

3.2 Prefix sum + HashMap (count/find subarrays matching a sum condition)

```
// Count subarrays whose sum equals k
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty prefix seen once -- crucial base case

    int runningSum = 0, count = 0;
    for (int num : nums) {
        runningSum += num;
        // if (runningSum - k) was seen before, that earlier prefix + this point = a valid subarray
        count += prefixCount.getOrDefault(runningSum - k, 0);
        prefixCount.merge(runningSum, 1, Integer::sum);
    }
    return count;
}
```

3.3 Difference array (many RANGE UPDATES, then read final array once)

```
// Apply many "add val to range [l, r]" updates efficiently, O(1) per update
public int[] rangeUpdates(int n, int[][] updates) {
    int[] diff = new int[n + 1];

    for (int[] u : updates) {
        int l = u[0], r = u[1], val = u[2];
        diff[l] += val;
        diff[r + 1] -= val; // cancel the effect after index r
    }

    int[] result = new int[n];
    int running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        result[i] = running; // this IS the prefix sum of diff -- rebuilds the final array
    }
    return result;
}
```

Difference array is prefix sum used BACKWARDS: instead of precomputing sums to answer range-sum queries fast, you record range-UPDATES in O(1) each, then run one prefix-sum pass at the end to materialize the final

array.

3.4 2D prefix sum (sub-rectangle sum queries)

```
public class NumMatrix {
    private int[][] prefix;

    public NumMatrix(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length;
        prefix = new int[rows + 1][cols + 1];
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                prefix[r+1][c+1] = matrix[r][c] + prefix[r][c+1] + prefix[r+1][c] - prefix[r][c];
            }
        }

        // sum of rectangle from (r1,c1) to (r2,c2) inclusive
        public int sumRegion(int r1, int c1, int r2, int c2) {
            return prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1];
        }
    }
}
```

The 2D formula's '+ prefix[r][c]' term corrects for double-subtracting the top-left overlapping region — classic inclusion-exclusion, same principle as counting the union of two overlapping sets.

4. Problem Bank — Tier 1 to Tier 6 (24 problems)

Tiers 1-2 build the basic range-query reflex; Tier 3 is the highest-leverage tier here (prefix sum + hashmap unlocks a huge class of counting problems) — don't skip it.

Tier 1 — Basic Range Sum Queries

LC#	Problem	Diff.	Key Concept / Theory
303	Range Sum Query - Immutable	Easy	Pure Section 3.1 template — build once, answer <code>rangeSum(l, r)</code> in $O(1)$ forever.
724	Find Pivot Index	Easy	Compare <code>leftSum</code> (prefix up to $i-1$) vs <code>totalSum - leftSum - nums[i]</code> at each index.
1480	Running Sum of 1d Array	Easy	The prefix array itself IS the answer — no query layer needed, just the build step.
1732	Find the Highest Altitude	Easy	Running prefix sum where the answer is simply the max prefix value reached.

Tier 2 — Prefix Sum as a Building Block

LC#	Problem	Diff.	Key Concept / Theory
238	Product of Array Except Self	Med	Prefix PRODUCT from the left + suffix product from the right, multiplied at each index — same accumulation idea, different operator.
485	Max Consecutive Ones	Easy	Not strictly prefix sum, but a good warmup in running-count resets adjacent to this pattern.
1685	Sum of Absolute Differences in a Sorted Array	Med	Prefix sum on a SORTED array lets you compute total absolute-difference contribution in $O(1)$ per index instead of $O(n)$.
2559	Count Vowel Strings in Ranges	Med	Build a prefix count array of a boolean condition (is-vowel-string), then answer each range query in $O(1)$.

Tier 3 — Prefix Sum + HashMap (counting subarrays)

LC#	Problem	Diff.	Key Concept / Theory
560	Subarray Sum Equals K	Med	THE canonical prefix-sum-plus-hashmap problem — Section 3.2's template, master this before anything else in this tier.
523	Continuous Subarray Sum	Med	Same idea but check remainders (<code>runningSum % k</code>) instead of raw sums — two indices with the same remainder means the subarray between them is divisible by k .
974	Subarray Sums Divisible by K	Med	Directly counts using the remainder-hashmap trick from 523 — handle negative remainders carefully in Java (<code>%</code> can return negative).
525	Contiguous Array	Med	Convert 0s to -1s, then it becomes 'subarray sum equals 0' — same hashmap trick as 560 with a clever input transformation.

Tier 4 — Difference Array (range UPDATE problems)

LC#	Problem	Diff.	Key Concept / Theory
1109	Corporate Flight Bookings	Med	Direct application of Section 3.3's difference array — many range-add updates, one final prefix-sum pass.
370	Range Addition	Med	Same difference array template, phrased generically instead of as a booking system.
1094	Car Pooling	Med	Difference array over a passenger-count timeline; check if any point exceeds capacity after the final prefix-sum pass.

Tier 5 — 2D Prefix Sum

LC#	Problem	Diff.	Key Concept / Theory
304	Range Sum Query 2D - Immutable	Med	Direct application of Section 3.4 — build once, answer sub-rectangle queries in $O(1)$.
1314	Matrix Block Sum	Med	2D prefix sum + clamping the query rectangle to the matrix boundaries for each cell's neighborhood.
1074	Number of Submatrices That Sum to Target	Hard	Combine 2D prefix sum (collapse rows between two boundaries) with the Tier 3 hashmap trick applied to each 1D collapsed row.

Tier 6 — Master Tier (combined techniques)

LC#	Problem	Diff.	Key Concept / Theory
327	Count of Range Sum	Hard	Prefix sum reduces this to 'count pairs of prefix values within a range' — solved with merge sort or a Binary Indexed Tree (Fenwick Tree) over prefix values, since a hashmap alone can't answer range-count queries efficiently.
1524	Number of Sub-arrays With Odd Sum	Med	Track counts of odd vs even prefix sums seen so far — a specialized case of the Tier 3 hashmap trick using parity instead of exact values.
1442	Count Triplets That Can Form Two Arrays of Equal XOR	Med	XOR prefix array (same idea as sum, but with XOR instead of +) — equal XOR between two prefixes means the XOR of everything between them is 0.
1738	Find Kth Largest XOR Coordinate Value	Med	2D XOR-prefix using the same inclusion-exclusion formula as Section 3.4, but with XOR replacing addition/subtraction throughout.

5. Fully Worked Examples (with reasoning)

Example A — 560. Subarray Sum Equals K (the canonical hashmap combo)

Problem: Count the number of contiguous subarrays whose sum equals k.

Reasoning

- If $\text{prefix}[j] - \text{prefix}[i] = k$ for some $i < j$, then the subarray between i and j sums to exactly k .
- Rearranged: $\text{prefix}[i] = \text{prefix}[j] - k$. So at each index j , ask: "how many earlier prefix sums equal $(\text{currentPrefixSum} - k)$?" — that count is exactly how many valid subarrays END at j .
- Store prefix sum FREQUENCIES in a HashMap as you go (not just existence — a value can repeat), and seed the map with $\{0: 1\}$ to correctly count subarrays that start at index 0.

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixFreq = new HashMap<>();
    prefixFreq.put(0, 1); // empty prefix (sum 0) has occurred once, before we start

    int runningSum = 0, count = 0;
    for (int num : nums) {
        runningSum += num;
        count += prefixFreq.getOrDefault(runningSum - k, 0);
        prefixFreq.merge(runningSum, 1, Integer::sum);
    }
    return count;
}
```

Example B — 238. Product of Array Except Self (prefix + suffix accumulation)

Problem: Return an array where $\text{output}[i] = \text{product of all elements except } \text{nums}[i]$, without using division.

Reasoning

- $\text{output}[i] = (\text{product of everything to the LEFT of } i) * (\text{product of everything to the RIGHT of } i)$.
- Compute the left-products in a first left-to-right pass (this IS a prefix product, same shape as prefix sum with $*$ instead of $+$).
- Compute the right-products in a second right-to-left pass, multiplying directly into the result array to keep $O(1)$ extra space (excluding the output array).

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];

    result[0] = 1;
    for (int i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1]; // prefix product up to i-1
    }

    int rightProduct = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= rightProduct; // multiply in suffix product
        rightProduct *= nums[i];
    }
    return result;
}
```

Example C — 304. Range Sum Query 2D (2D prefix sum, inclusion-exclusion)

Problem: Answer many sub-rectangle sum queries on a fixed 2D matrix.

Reasoning

- Build $\text{prefix}[r][c]$ = sum of the rectangle from (0,0) to (r-1, c-1) — same off-by-one shift as 1D, applied to both dimensions.
- To get a specific sub-rectangle sum, take the full rectangle up to the bottom-right corner, subtract the two rectangles above and to the left of the target region, then ADD BACK the top-left corner rectangle that got subtracted twice — classic inclusion-exclusion.

```
// prefix built in constructor (see Section 3.4)
public int sumRegion(int row1, int col1, int row2, int col2) {
    return prefix[row2+1][col2+1] - prefix[row1][col2+1] - prefix[row2+1][col1] + prefix[row1][col1];
}
```

Example D — 1109. Corporate Flight Bookings (difference array)

Problem: Given many [first, last, seats] booking updates over n flights, return the total seats booked per flight.

Reasoning

- Applying each update directly (looping first to last, adding seats) is $O(\text{range})$ per update — $O(n \cdot q)$ total for q updates. Too slow for many updates.
- Instead, record the EFFECT of each update in $O(1)$: $\text{diff}[\text{first}] += \text{seats}$ (start adding here) and $\text{diff}[\text{last}+1] -= \text{seats}$ (stop adding after here).
- Run ONE prefix-sum pass over $\text{diff}[]$ at the end — the running total at each index IS the final answer for that flight, because the +seats and -seats markers correctly turn on/off exactly within [first, last].

```
public int[] corpFlightBookings(int[][] bookings, int n) {
    int[] diff = new int[n + 1];

    for (int[] b : bookings) {
        int first = b[0] - 1, last = b[1] - 1, seats = b[2]; // convert to 0-indexed
        diff[first] += seats;
        diff[last + 1] -= seats;
    }

    int[] result = new int[n];
    int running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        result[i] = running;
    }
    return result;
}
```

6. The Variant Family — Map of Every Disguise

Variant	What Changes	Example
Standard 1D range sum	Base case, subtraction formula	LC 303
Prefix + HashMap	Count/find subarrays by target sum (not just query a fixed range)	LC 560, 523, 974
Prefix product / other ops	Same accumulation shape, different operator (*, XOR)	LC 238, 1442
Difference array	Reversed use-case: many range UPDATES, one final read	LC 1109, 370, 1094
2D prefix sum	Inclusion-exclusion across two dimensions	LC 304, 1314
2D + hashmap combo	Collapse one dimension via prefix sum, then apply Tier 1 on the result	LC 1074
Prefix + Fenwick/merge sort	When you need RANGE counts of prefix values, not just a single match	LC 327

Same exercise as the Kadane's variant table: before coding a new problem, try to place it in one of these rows. If it doesn't fit any row, it might not actually be a prefix sum problem.

7. Common Mistakes

- **Forgetting `prefix[0] = 0` (the empty-prefix base case):** breaks range sums that start at index 0, AND breaks the HashMap-based counting trick (560) for subarrays that start at index 0 — always seed the map with `{0: 1}`.
- **Off-by-one in the subtraction formula:** `rangeSum(left, right) = prefix[right+1] - prefix[left]`, NOT `prefix[right] - prefix[left]`. Writing this on paper with a tiny example before coding avoids the whole class of bugs.
- **Using existence (HashSet) instead of frequency (HashMap<Integer,Integer>) in counting problems:** the same prefix sum value can occur multiple times — 560 needs COUNTS, not just 'have I seen this before'.
- **Negative modulo bug in Java (523, 974):** Java's `%` can return a negative result for negative inputs (e.g. `-3 % 5 == -3`, not 2). Normalize with `((sum % k) + k) % k` before using it as a hashmap key.
- **Applying prefix sum when the array is mutable/frequently updated:** a plain prefix sum array is $O(n)$ to REBUILD after any single-element update — if updates are frequent, you need a Binary Indexed Tree (Fenwick Tree) or Segment Tree instead (See LC 307, a natural next topic).
- **Integer overflow on large arrays:** prefix sums can grow large fast — use long instead of int when array values or lengths are large enough that the cumulative sum could exceed `Integer.MAX_VALUE`.
- **2D prefix sum inclusion-exclusion sign errors:** forgetting the `'+ prefix[r1][c1]'` add-back term double-subtracts the top-left overlap — always derive this formula from a small 2x2 example rather than memorizing it blindly.

8. Practice Roadmap & Spaced Repetition

Suggested order (2-3 problems/day)

- **Days 1-2:** Tier 1 (all 4 problems) — build comfort with the off-by-one convention and the $O(1)$ query formula.
- **Days 3-5:** Tier 2 (all 4 problems) — prefix used as a building block inside a larger computation, not just for direct queries.
- **Days 6-9:** Tier 3 (all 4 problems) — the highest-leverage tier; master 560 completely before touching 523/974/525.
- **Days 10-12:** Tier 4 (all 3 problems) — difference array, the 'reversed' use of this pattern.
- **Days 13-15:** Tier 5 (all 3 problems) — 2D extension; derive the inclusion-exclusion formula yourself before checking Section 3.4.
- **Days 16-19:** Tier 6 — attempt cold, these combine prefix sum with hashmaps, XOR, or advanced structures.

Spaced repetition after first solve

- Day 1: solve with full struggle (aim 20-25 min before hints).
- Day 3: re-solve cold — for Tier 3 problems, check you can state the ' $\text{prefix}[i] = \text{prefix}[j] - k$ ' rearrangement from memory.
- Day 7: re-solve cold again; note which variant-table row (Section 6) the problem belongs to.
- Day 21: final cold re-solve of 303, 560, and 304 back to back — if all three are fast and error-free, the core pattern (and both its 1D/2D forms) is internalized.

Same method as your Sliding Window, Two Pointers, ArrayList, and Kadane's guides — struggle first, use this only after a genuine attempt.